EXPENSE TRACKER

Python Console Application

Project Report

Submitted by: HARSH GODARA

Reg. No.: 26BCE10980

Course : B.Tech – CSE CORE

Submitted to – Dr. AVR MAYURI

SLOT – A21 + A22 + A23 + D21 + D22 + B14

A command-line application for recording, analyzing, and monitoring personal expenses.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Workflow
8. Data Storage Design

9. Implementation Details
10. Testing Approach and Results
11. Screenshots / Sample Results
12. Design Decisions
13. Challenges Faced
14. Learnings
15. Limitations
16. Future Enhancements
17. Conclusion
18. References

Introduction

Expense Tracker is a command-line Python application designed to help users maintain a simple record of everyday expenses. It provides expense entry, viewing, total-spending calculation, category-wise analysis, highest-expense identification, and monthly budget monitoring. Expense records are stored locally in a text file.

Problem Statement

Manual tracking of daily expenses can make it difficult to understand spending patterns and monitor a planned budget. The project addresses this problem with a lightweight local application that records expenses and produces basic spending summaries through a menu-driven interface.

Objectives

• Record personal expenses with a name, amount, and category.

• Maintain expense records using local file storage.

• Calculate total spending automatically.

• Group spending by category.

• Identify the highest individual expense.

• Set and monitor a monthly budget.

• Handle invalid input without terminating the program.

Functional Requirements

No.	Requirement
1	Add an expense.
2	View all expenses.
3	Calculate total spending.
4	Calculate category-wise spending.
5	Find highest expense.
6	Set monthly budget.
7	View budget status.
8	Load/save expense records.

Non-Functional Requirements

• Usability: simple menu-driven interface.

• Reliability: invalid numeric input is handled using exception handling.

• Maintainability: repeated operations are organized into functions.

• Persistence: expenses remain available after restarting the program.

• Error handling: missing files, invalid values, and empty fields are handled.

System Architecture

The application uses a simple command-line architecture. The user interacts with the menu, the program validates and processes the requested operation, and expense records are stored in a local text file.

User / Command Line
↓
Expense Tracker Application
Menu • Validation • Processing • Reports • Budget
↓
expense.txt — Local File Storage

Workflow

• Start the program.

• Load existing expenses from expense.txt.

• Display the main menu.

• Read and validate the user's menu choice.

• Perform the selected operation.

• Save newly added expenses when required.

• Display the result and return to the menu.

• Exit when option 8 is selected.

Data Storage Design

The project uses a plain text file named expense.txt. Each expense is stored on one line using the delimiter '|'. The fields are expense name, amount, and category.

Field	Example	Purpose
Expense Name	College Canteen	Identifies the expense.
Amount	120.0	Stores the monetary value.
Category	FOOD	Groups related expenses.

Implementation Details

• Lists store expense records in memory.

• A dictionary is used for category-wise totals.

• calculate_total() calculates total spending.

• save_expenses() writes records to expense.txt.

• Loops process records and generate reports.

• Conditional statements control menu operations and budget status.

• try/except blocks validate numeric input and handle file issues.

Testing Approach

Test	Condition	Result
Add expense	Valid values	Pass
Invalid menu	abc / 2d	Pass

Invalid amount	abc	Pass
Empty name	Blank input	Pass
Empty category	Blank input	Pass
Zero/negative amount	0 / -100	Pass
Total	Five sample expenses	₹900
Categories	Four categories	Correct totals
Highest	Five sample expenses	Movie, ₹250
Budget	₹2000 budget	₹1100 remaining
Persistence	Restart program	Records loaded

Sample Results/ Screenshots

Final test data: College Canteen 120 (FOOD), Bus Pass 200 (TRAVEL), Notebook 180 (EDUCATION), Movie 250 (ENTERTAINMENT), and Lunch 150 (FOOD). Total spending was 900. Category totals were FOOD 270, TRAVEL 200, EDUCATION 180, and ENTERTAINMENT 250. The highest expense was Movie at 250. With a budget of 2000, 1100 remained.

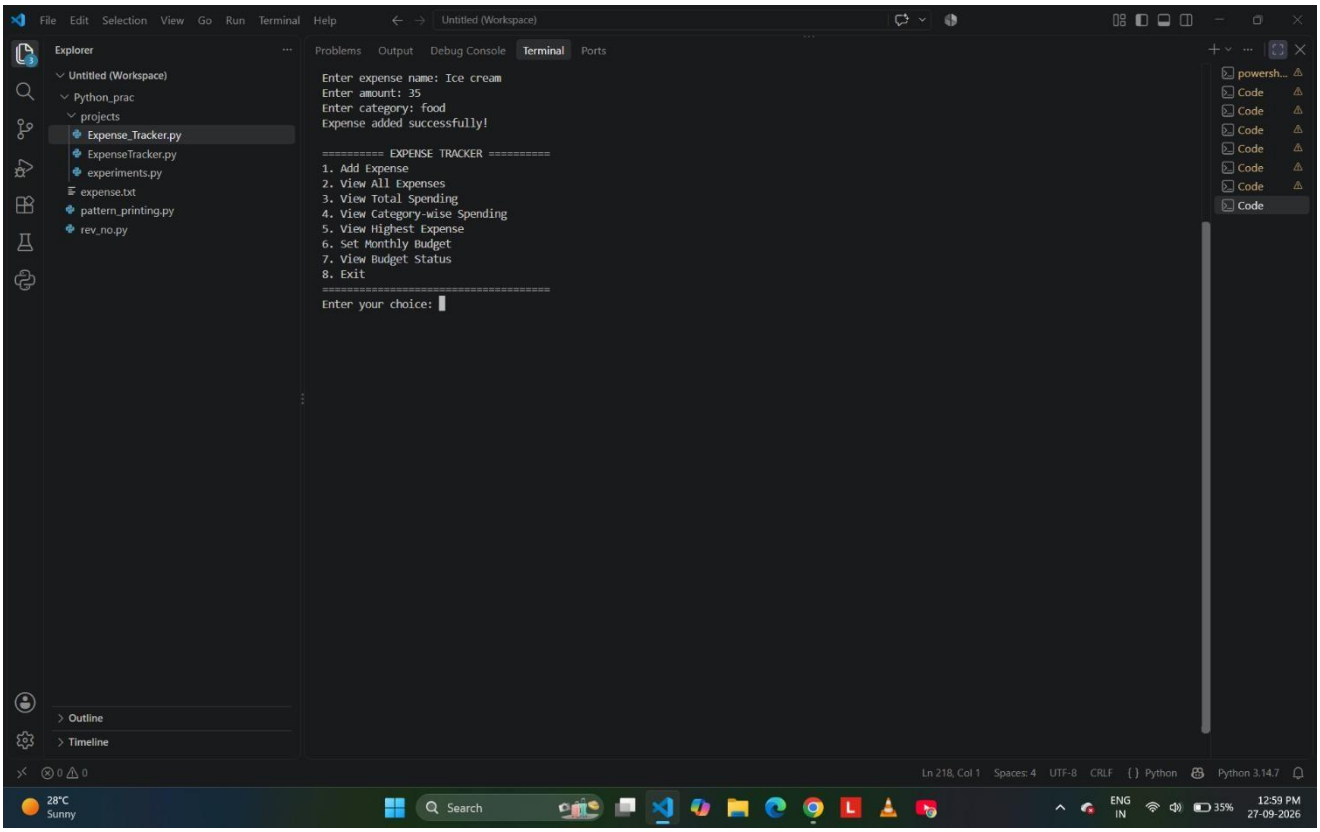


Fig.1 Adding a new expense

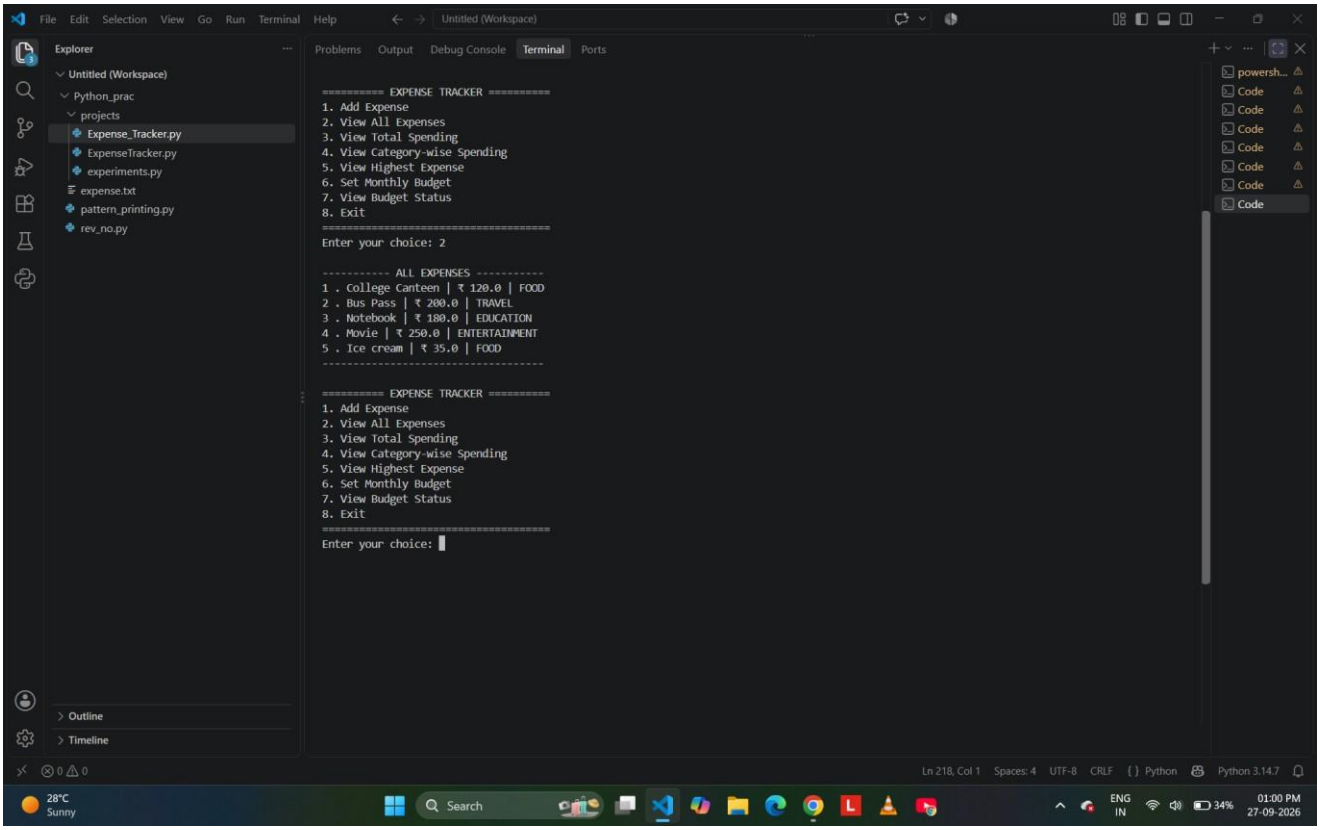


Fig.2 Displaying all expenses

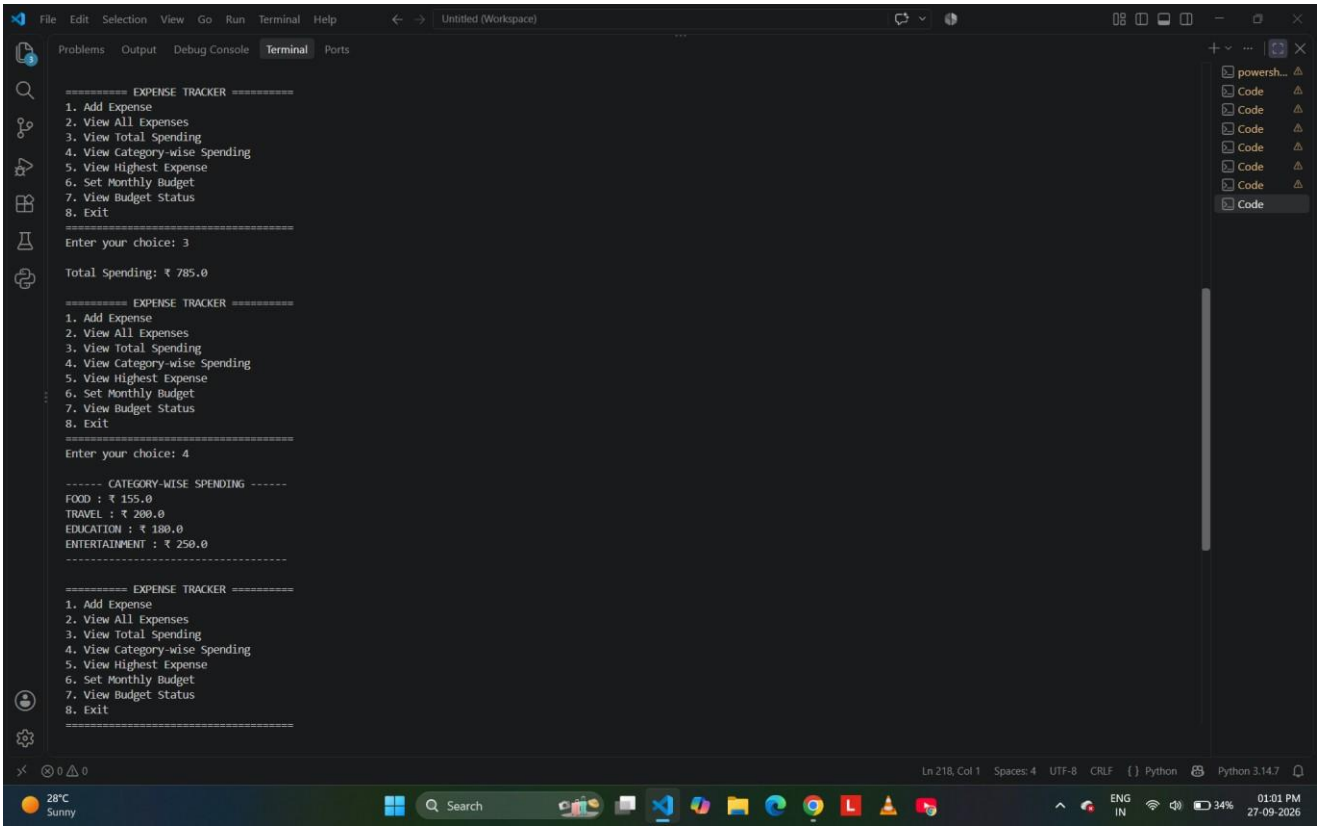


Fig.3 Spending Analysis

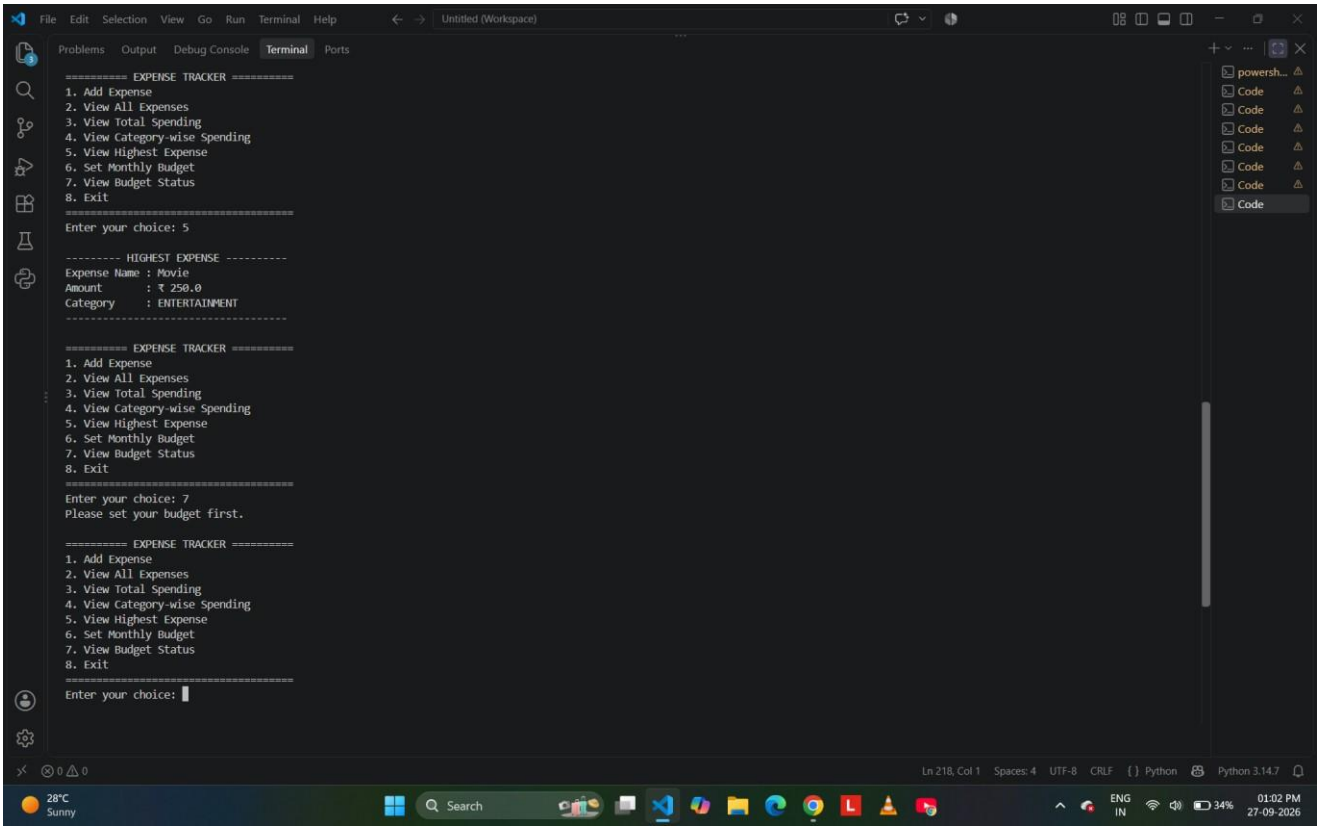


Fig.4 Testing

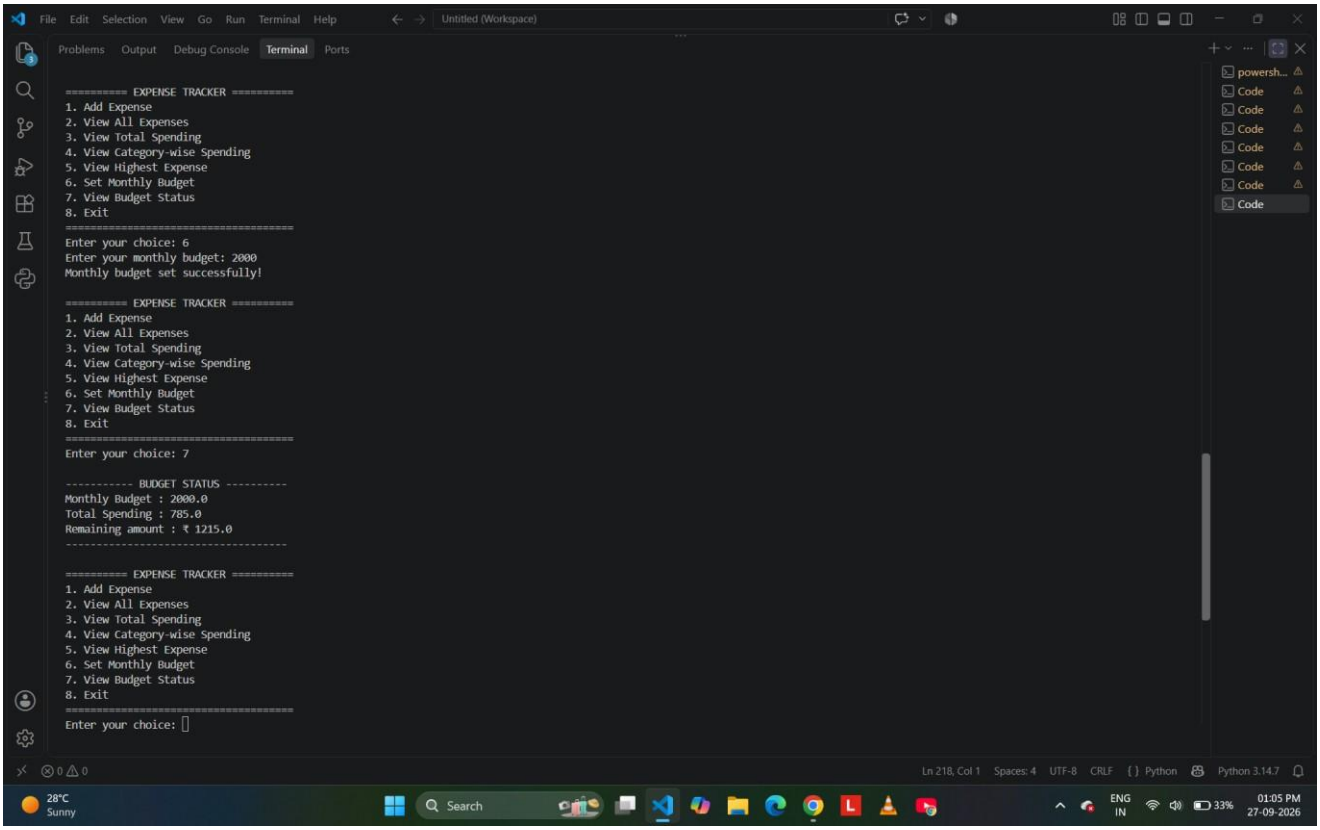


Fig.5 Budget Analysis

Design Decisions and Rationale

- A command-line interface keeps the project focused on Python fundamentals and avoids externaldependencies.
- A text file provides simple, transparent persistence for a small student project.
- Lists demonstrate basic Python data structures for storing records.
- A dictionary supports dynamic category aggregation without hard-coded categories.
- Functions separate reusable operations such as total calculation and file saving.
- Input validation improves reliability during real use.

Challenges Faced

- Managing multiple structured expense records.
- Saving and loading structured data from a text file.
- Handling invalid numeric input without crashing.
- Calculating dynamic category totals.
- Combining multiple Python concepts into one practical program.

Learnings

- Using lists to manage collections of records.
- Using dictionaries for grouping and aggregation.
- Using functions for reusable logic.
- Using file handling for persistence.
- Using exception handling for invalid input.
- Building a menu-driven application from basic Python concepts.

Limitations

- The application is command-line based.
- The monthly budget is maintained only for the current program session.
- Text-file storage is intended for small datasets.
- There is no authentication or multi-user support.

Future Enhancements

- Add date/month information.
- Add search and filtering.

- Generate monthly reports.
- Export to CSV.
- Add spending charts.
- Use database storage.
- Persist monthly budgets.

Conclusion

The Expense Tracker demonstrates how fundamental Python programming concepts can be combined to create a useful command-line application. The final program supports expense recording, persistent storage, spending analysis, highest-expense identification, budget monitoring, and input validation. Testing with both valid and invalid inputs showed that the application can perform its intended operations while handling common user errors safely.

References

Python documentation and course learning materials were used for Python syntax, file handling, functions, lists, dictionaries, loops, and exception handling. The VITyarthi project requirements were used to structure the project documentation and submission materials.